

Guía de Ejercicios Propuestos: Patrones de Diseño Fundamentales

Introducción: ¡Bienvenido a tu viaje en el mundo de los patrones de diseño! Estos ejercicios están diseñados para ayudarte a comprender no solo qué son los patrones, sino también por qué son cruciales para escribir código de calidad, flexible y mantenible. A través de la práctica, verás cómo los patrones proporcionan **soluciones probadas a problemas comunes en el diseño de software**, mejorando tu forma de abordar la programación orientada a objetos.

Ejercicios sobre Conceptos Fundamentales de Patrones

Ejercicio 1: Comprendiendo los Patrones de Diseño

- **Objetivo de Aprendizaje:** Comprender la definición y la importancia de aprender sobre patrones de diseño.
- **Contexto/Descripción del Problema:** Las fuentes indican que es posible programar durante años sin conocer los patrones, pero que aprenderlos ofrece beneficios significativos.
- **Instrucciones Específicas:**
 1. Basándote en la sección "¿Por qué debería aprender sobre patrones?", **explica con tus propias palabras qué es un patrón de diseño.**
 2. Menciona **al menos dos razones clave por las que un desarrollador de software debería dedicar tiempo a aprender sobre ellos**, incluso si ya puede codificar sin conocerlos explícitamente.
 3. Considera el concepto de "**lenguaje común**" que los patrones proporcionan.
- **Pistas/Consejos:** Piensa en los patrones como un "juego de herramientas de soluciones comprobadas" y cómo facilitan la comunicación en un equipo de desarrollo.
- **Material de Referencia:** Sección "Patrones: ¿Por qué debería aprender sobre patrones?".

Ejercicio 2: Diferenciando Patrones y Algoritmos

- **Objetivo de Aprendizaje:** Distinguir claramente entre el concepto de un patrón de diseño y un algoritmo.
- **Contexto/Descripción del Problema:** A menudo, los patrones se confunden con algoritmos debido a que ambos describen soluciones típicas a problemas conocidos. Sin embargo, son fundamentalmente distintos.
- **Instrucciones Específicas:**
 1. Describe la **diferencia principal entre un patrón de diseño y un algoritmo** según la guía.
 2. Utiliza la **analogía proporcionada en la guía (receta de cocina y plano)** para explicar cómo cada uno aborda la solución de un problema.

- **Pistas/Consejos:** Un algoritmo define un "grupo claro de acciones para lograr un objetivo", mientras que un patrón es una "descripción de más alto nivel de una solución".
- **Material de Referencia:** Sección "Patrones", párrafos sobre la confusión entre patrones y algoritmos.

Ejercicio 3: Clasificación y Tipos de Patrones de Diseño

- **Objetivo de Aprendizaje:** Identificar las diferentes clasificaciones de patrones y los tipos de patrones de diseño por su propósito.
- **Contexto/Descripción del Problema:** Los patrones se clasifican por su complejidad, nivel de detalle y escala de aplicabilidad. Dentro de los patrones de diseño, existen clasificaciones por propósito.
- **Instrucciones Específicas:**
 1. Enumera y describe brevemente los **tres niveles de patrones** mencionados en la guía (Idioms, Patrones de Diseño, Patrones de Arquitectura).
 2. Luego, enfócate en los **patrones de diseño** y describe los **tres tipos principales según su propósito** (Creacionales, Estructurales, de Comportamiento), explicando brevemente lo que cada tipo aborda.
- **Pistas/Consejos:** Recuerda que los patrones de diseño son soluciones probadas y comunes para problemas de diseño de software. La guía se centra en los patrones de diseño.
- **Material de Referencia:** Sección "Clasificación de los patrones".

Ejercicios sobre Patrones Creacionales en Java

Ejercicio 4: Patrón Singleton: Concepto y Utilidad

- **Objetivo de Aprendizaje:** Comprender el propósito principal del patrón Singleton y su aplicación fundamental.
- **Contexto/Descripción del Problema:** El patrón Singleton es uno de los más conocidos y garantiza que una clase tenga una única instancia. Es el patrón central de la actividad formativa.
- **Instrucciones Específicas:**
 1. Basado en la definición proporcionada, **explica qué garantiza el patrón Singleton.**
 2. ¿Por qué sería útil este patrón en un escenario como el `DiscountManager` propuesto en la actividad, donde se requiere un sistema de descuentos único para una tienda de ropa?.
 3. Menciona las **dos características clave recomendadas** para su implementación (constructor y método `getInstance()`).
- **Pistas/Consejos:** Piensa en la necesidad de tener un "punto de acceso global" a una instancia específica.
- **Material de Referencia:** Sección "4. Singleton", y "Descripción de la actividad".

Ejercicio 5: Implementación Básica del Patrón Singleton en Java

- **Objetivo de Aprendizaje:** Implementar la estructura básica de una clase que siga el patrón Singleton en Java.
- **Contexto/Descripción del Problema:** La actividad formativa pide implementar el patrón Singleton para la clase `DiscountManager`. Este ejercicio te guiará en los pasos esenciales para una clase genérica.
- **Instrucciones Específicas:**
 1. Crea una nueva clase Java llamada `ConfiguracionSistema`.
 2. Implementa el patrón Singleton en esta clase, asegurando que:
 - Tenga un **constructor privado**.
 - Declare una **instancia privada, estática y final de la propia clase**.
 - Cree un **método estático `getInstance()`** que devuelva la única instancia de la clase.
 3. Añade un método simple, por ejemplo, `obtenerVersion()` que devuelva un `String` como "Versión 1.0".
 4. En un método `main` (en otra clase o en la misma con fines de prueba), obtén la instancia de `ConfiguracionSistema` dos veces usando `getInstance()` y comprueba que ambas referencias apuntan a la misma instancia (puedes usar `==` o imprimir el `hashCode()`).
- **Pistas/Consejos:** Presta atención a las instrucciones del "Paso 2" de la actividad y a los criterios de evaluación formativa.
- **Material de Referencia:** "Instrucciones específicas: Paso 2", "Pauta de evaluación formativa".

Ejercicio 6: Patrón Factory Method: Concepto y Flexibilidad

- **Objetivo de Aprendizaje:** Comprender el propósito del patrón Factory Method y cómo mejora la flexibilidad en la creación de objetos.
- **Contexto/Descripción del Problema:** El Factory Method permite crear objetos sin especificar la clase exacta que se creará, delegando esta responsabilidad a las subclases.
- **Instrucciones Específicas:**
 1. Explica **qué problema resuelve el patrón Factory Method** en el desarrollo de software.
 2. ¿Cómo facilita este patrón la **adición de nuevas clases de objetos** sin modificar el código existente que los utiliza?.
 3. Proporciona un ejemplo hipotético (no de las fuentes) de un escenario donde el Factory Method sería beneficioso, como una aplicación de procesamiento de documentos que puede crear diferentes tipos de documentos (PDF, Word, etc.).
- **Pistas/Consejos:** El Factory Method es también conocido como "Método fábrica" o "Constructor virtual". Su objetivo es "desactivar el código que crea los objetos del código que los utilizan".
- **Material de Referencia:** Sección "1. Factory Method".

Ejercicio 7: Implementación Simple del Patrón Factory Method

- **Objetivo de Aprendizaje:** Aplicar el patrón Factory Method para crear objetos de manera flexible.
- **Contexto/Descripción del Problema:** Para entender el Factory Method, es útil ver cómo se implementan las clases `Creator`, `ConcreteCreator`, `Product` y `ConcreteProduct`.
- **Instrucciones Específicas:**
 1. Imagina que estás desarrollando un sistema para una tienda de mascotas que vende diferentes tipos de `Animal` (por ejemplo, `Perro`, `Gato`).
 2. Crea una interfaz o clase abstracta `Animal` con un método `emitirSonido()`.
 3. Implementa clases concretas `Perro` y `Gato` que implementen `Animal`.
 4. Crea una clase abstracta `CreadorAnimal` con un método abstracto `crearAnimal()`.
 5. Implementa clases concretas `CreadorPerro` y `CreadorGato` que extiendan `CreadorAnimal` y sobrescriban `crearAnimal()` para devolver una instancia de `Perro` y `Gato` respectivamente.
 6. En un método `main`, utiliza los `CreadorPerro` y `CreadorGato` para crear y hacer que los animales emitan su sonido, demostrando que puedes crear diferentes tipos de animales a través de la interfaz del creador.
- **Pistas/Consejos:** Sigue la estructura de ejemplo proporcionada en las Figuras 5, 6 y 7 de la guía.
- **Material de Referencia:** Sección "Ejemplo de código" para Factory Method.

Ejercicio 8: Patrón Abstract Factory: Familias de Objetos Relacionados

- **Objetivo de Aprendizaje:** Comprender cuándo y por qué utilizar el patrón Abstract Factory para crear familias de objetos relacionados.
- **Contexto/Descripción del Problema:** El Abstract Factory se usa cuando necesitas agrupar un conjunto de clases que tienen un funcionamiento en común, conocidas como "familias", especialmente cuando puede haber implementaciones paralelas de estas familias.
- **Instrucciones Específicas:**
 1. **Define el propósito principal del patrón Abstract Factory.**
 2. Explica la **diferencia clave entre el Factory Method y el Abstract Factory**. ¿Qué tipo de problemas resuelve cada uno?
 3. Basándote en el ejemplo del "simulador de tienda de muebles" mencionado en la guía, o creando uno propio, **describe un escenario donde el Abstract Factory sería la solución adecuada**, enfocándote en la creación de "familias de objetos relacionados o dependientes".
- **Pistas/Consejos:** El Abstract Factory "proporciona una interfaz para crear familias de objetos relacionados o dependientes sin especificar sus clases concretas".
- **Material de Referencia:** Sección "2. Abstract Factory".

Ejercicio 9: Patrón Constructor (Builder): Construcción Paso a Paso

- **Objetivo de Aprendizaje:** Entender el propósito del patrón Builder y cuándo es ventajoso para construir objetos complejos.
- **Contexto/Descripción del Problema:** El patrón Builder es útil para construir objetos complejos paso a paso, separando el proceso de construcción de la representación final del objeto.
- **Instrucciones Específicas:**
 1. **Explica en qué consiste el patrón Builder** y cuál es su principal ventaja.
 2. Proporciona un **ejemplo de un objeto complejo (no de los proporcionados en las fuentes, por ejemplo, una Pizza con muchos ingredientes, un Reporte con secciones variables, etc.)** que sería difícil de construir con un constructor tradicional debido a la gran cantidad de parámetros opcionales o a la variabilidad en su construcción.
 3. Describe **cómo el patrón Builder simplificaría la creación** de este objeto complejo en tu ejemplo.
- **Pistas/Consejos:** El Builder permite "crear diferentes representaciones de un objeto utilizando el mismo proceso de construcción".
- **Material de Referencia:** Sección "3. Constructor (Builder)".

Ejercicio 10: Aplicación Detallada del Patrón Singleton (Caso DiscountManager)

- **Objetivo de Aprendizaje:** Aplicar el patrón Singleton al caso práctico propuesto en la actividad formativa, reforzando la implementación de sus componentes clave.
- **Contexto/Descripción del Problema:** Retomando el escenario de la actividad formativa, debes desarrollar un sistema de pedidos online para una tienda de ropa que requiere un sistema de descuentos único (*DiscountManager*).
- **Instrucciones Específicas:**
 1. En un proyecto Java (utilizando **Apache NetBeans** y **JDK**, como se recomienda), crea una clase llamada *DiscountManager*.
 2. Implementa el patrón Singleton en esta clase, asegurando que cumpla con los **tres criterios de evaluación**:
 - El **constructor de *DiscountManager* debe ser privado**.
 - Debe declarar una **instancia privada, estática y final** de la propia clase (ej. `private static final DiscountManager instance = new DiscountManager();`).
 - Debe tener un **método estático `getInstance()`** que devuelva la única instancia de *DiscountManager*.
 3. Añade un método `applyDiscount(double price, String productType)` a la clase *DiscountManager* que simplemente imprima un mensaje simulando la aplicación de un descuento (ej., "Aplicando descuento del 15% a " + `productType` + " (Precio original: " + `price` + ")").
 4. En tu método `main`, demuestra el funcionamiento del Singleton:
 - Obtén la instancia de *DiscountManager* varias veces (ej., `DiscountManager dm1 = DiscountManager.getInstance();`, `DiscountManager dm2 = DiscountManager.getInstance();`).
 - Llama al método `applyDiscount()` usando ambas referencias (`dm1` y `dm2`) y verifica que se está operando sobre la misma instancia.

- **Pistas/Consejos:** Este ejercicio es una implementación directa de los pasos 1 y 2 de la actividad formativa. Utiliza los criterios de evaluación como una lista de verificación.
 - **Material de Referencia:** "Descripción de la actividad", "Instrucciones específicas: Paso 2", "Pauta de evaluación formativa".
-